

Ошибки логики

Описание проблемы

Бизнес-логика определяет поведение приложения в любых возможных сценариях его использования. В свою очередь, уязвимости бизнес-логики связаны с недостатками в архитектуре или ошибками, допущенными при реализации логики приложения.

Отличие уязвимостей бизнес-логики от других:

- они индивидуальны для каждого приложения и зависят от него;
- нет единой причины появления ошибок бизнес-логики, как нет и единого шаблона их обнаружения. Это и делает поиск подобных уязвимостей сложным;
- они предполагают манипуляции легитимной функциональностью приложений.

Причины возникновения ошибок логики:

- недостаточные требования к входным данным;
- неполная информация о предметной области и компонентах приложения;
- неверные предположения о действиях пользователя;
- избыточное доверие клиентской части приложения.

Небезопасные прямые ссылки на объекты

Частным случаем ошибок логики являются небезопасные прямые ссылки на объекты (IDOR). Такие уязвимости появляются, когда разработчик предполагает, что пользователь, увидев сообщение об отсутствии прав на интерфейсе, не предпримет попытку вызвать API напрямую. Одновременно, запрос, который пытается отправить пользователь, имеет в качестве параметра целочисленный идентификатор, позволяющий получить информацию об объекте.

Например, у нас есть метод, позволяющий изменить текст задачи Документ.WriteForParticipant.

Вот его сигнатура:

```
1  {
2      "jsonrpc": "2.0",
3      "method": "СлужЗап.WriteForParticipant",
4      "params": {
5          "Запись": {
6              "f": 0,
7              "d": [
8                  215091264,
9                  "[[\"p\", {\"version\": \"2\"}, \"changed
text\"]]",
10                 true
11             ],
12             "s": [
13                 {
14                     "n": "@Документ",
15                     "t": "Число целое"
16                 },
17                 {
18                     "n": "РазличныеДокументы.ИнформацияJSON",
19                     "t": "Строка"
20                 },
21                 {
22                     "n": "РучноеСохранение",
```

```

23                                     "t": "Логическое"
24                                     }
25                                 ],
26                                 "_type": "record"
27                             },
28                             "ДопПоля": { "ИмяОбъекта": "СлужЗап" }
29                         },
30                         "protocol": 6,
31                         "id": 1
32 }

```

Пользователь может узнать запрос, изменив любую свою задачу. Затем просто начать перебирать первый параметр @Документ и, таким образом, изменить текст произвольных задач, если проверка прав реализована только на клиенте.

Последствия эксплуатации

Ошибки логики приводят к взаимодействию с приложением по сценарию, который не предусмотрен разработчиком. Результат выполнения таких сценариев может привести к таким последствиям, как:

- несанкционированное изменение чужих данных.
- эксплуатация легитимных механизмов приложения для извлечения собственной выгоды. Например, классическая история, когда покупатель может купить себе товар за 1 рубль или майнить бонусные баллы.
- получение конфиденциальных данных компании, которые могут использоваться для последующей атаки на инфраструктуру.

Как исправить?

1. Не доверять клиентской части приложения и проводить [проверки](#) (не только безопасности, но и логики) на серверной стороне.
2. Тщательно прорабатывать сценарии работы с системой, помните, что пользователь может отклоняться от задуманного вами бизнес-процесса.
3. Не использовать целочисленные идентификаторы, которые злоумышленник может перебрать, для доступа к объектам системы.